

E-Commerce Sales Analytics on ClickHouse

DSCI 551 Course Project — Phase 3: Final Report

Team Members: Wenxin Li, Isabelle Du, Xinyuan Huang

Date: May 8th, 2026

1. Introduction and Motivation

Many e-commerce analytics queries follow the same pattern: scan millions of order rows, touch two or three columns (price, quantity, maybe region or date), and return a small aggregated result. In row-oriented systems such as MySQL's InnoDB, all columns of a row are stored together in the same 16 KB page, so even a query that only needs price and quantity may still read unrelated fields such as category, city, and payment_method (Abadi et al., 2008). ClickHouse addresses this problem with columnar storage, so a query such as `sum(price * quantity)` reads only the relevant column files instead of the full row.

We chose ClickHouse because it is designed for analytical workloads that repeatedly aggregate a small number of columns across very large datasets. Its columnar layout, vectorized execution model, and built-in distributed query support make it a strong match for an e-commerce sales analytics application (Schulze et al., 2024).

The project builds a command-line analytics tool on top of ClickHouse and runs it against one million synthetic e-commerce orders. Sixteen features cover different pieces of the engine — columnar scans, partition pruning, sparse indexes, bloom filter and minmax skip indexes, vectorized pipelines, HyperLogLog sketches, batch inserts, and two-shard distributed aggregation. For each feature, the report traces the SQL statement down to the engine's internal access path and explains what makes the query fast or slow.

The dataset spans January 2023 through December 2024 and includes six world regions, thirty cities, eight product categories, five payment methods, and 100,000 distinct user IDs. Prices are drawn uniformly between \$1 and \$500, and quantities range from 1 to 10.

2. Database System Overview

ClickHouse was originally developed by Yandex around 2009 as the engine behind Metrika, its web analytics service. The codebase was released as open source in June 2016 under the Apache 2.0 license, and maintenance was later taken over by ClickHouse, Inc. in 2021. Schulze et al. (2024) provided the first peer-reviewed description of the system’s architecture.

Two design choices set ClickHouse apart from general-purpose databases. First, it stores data in columnar files. Each column of a table part occupies its own file on disk, compressed with LZ4 by default. As a result, a query such as `sum(price * quantity)` reads only the price and quantity columns rather than loading unrelated attributes such as category or city into memory. Second, the query engine processes data in vectors, or batches of up to 8,192 values per column, instead of row by row. These tight loops over same-type arrays improve cache efficiency and reduce per-row function-call overhead.

3. Internal Architecture

3.1 Storage — MergeTree Engine

MergeTree is the default and most common storage engine in ClickHouse. When a client sends an INSERT, the engine sorts the incoming rows by the ORDER BY columns, writes them as an immutable “part” on disk, and returns. A background thread wakes up periodically, picks a few small parts, and merges them into a bigger one. The pattern looks like an LSM tree, but ClickHouse treats all parts as equals and does not organize them into a level hierarchy (Schulze et al., 2024).

Partitioning adds another organizational layer. In our project, the table definition uses `PARTITION BY toYYYYMM(order_date)`, which groups data into one partition per calendar month. Parts from different partitions never get merged together. After loading one million rows, shard 1 contains 24 partitions (January 2023 through December 2024), each holding between 19,000 and 21,500 rows. February partitions are slightly smaller because the month has fewer days, while the remaining partitions stay relatively uniform in size. **Figure 1** shows the row count and disk size for each monthly partition on shard 1.

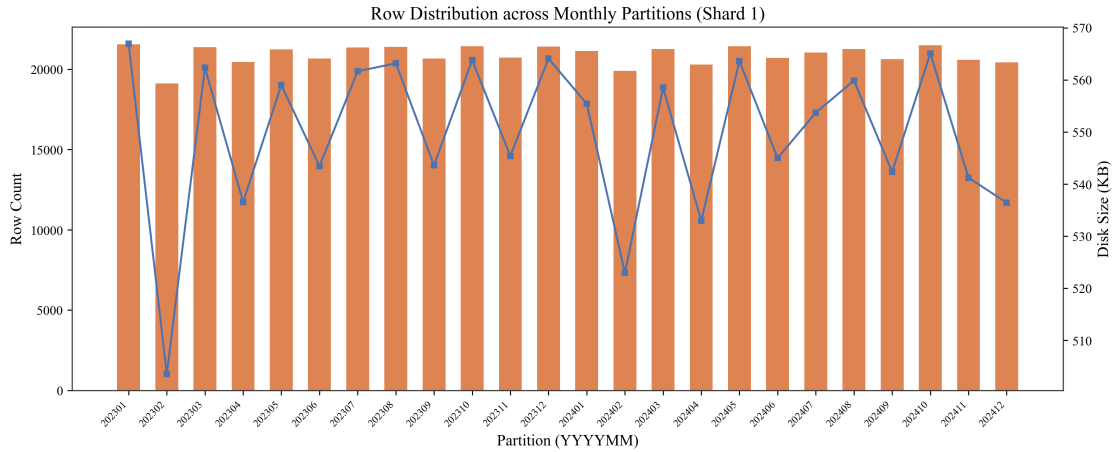


Figure 1. Row count and disk size per monthly partition on shard 1

Compression works at the column level. Each column file is compressed independently, so the ratio depends on the data distribution within that column. The region column contains only six distinct string values, and because it is the first column in the ORDER BY key, identical values cluster together in long runs on disk. LZ4 compresses those runs at about 23.4 $\times$. In contrast, the price column contains near-random floating-point values and compresses at about 1.8 $\times$. The order_date column uses the Date type, and because values repeat often within each monthly partition, it reaches about 9.7 $\times$ compression.

Feature 12 of the application reads these counters from system.parts_columns. **Figure 2** compares compressed and uncompressed column sizes and shows how compression ratios vary across columns.

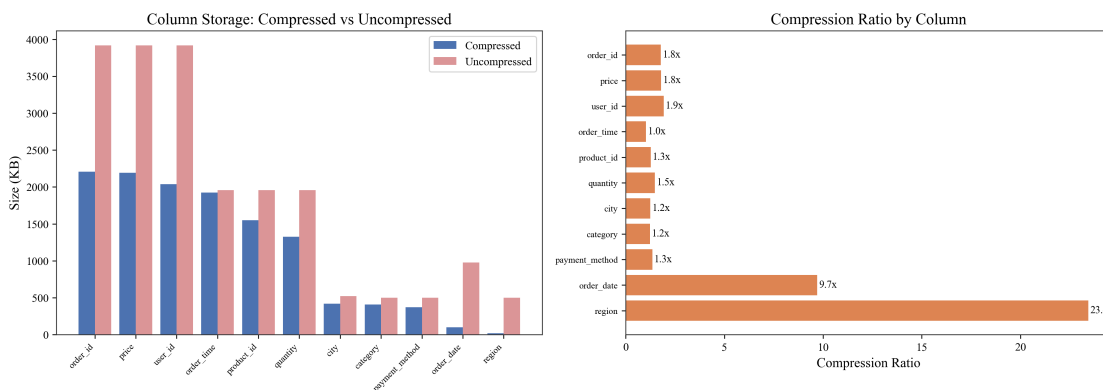


Figure 2. Compressed vs. uncompressed column sizes and compression ratios

3.2 Indexing — Sparse Index and Skip Indexes

InnoDB in MySQL builds a dense B+ tree in which every row receives an index entry. ClickHouse takes a different approach. MergeTree’s primary index is sparse, storing one key sample per granule of 8,192 rows (ClickHouse Documentation, n.d.). At query time, the engine binary-searches those samples to find which granules may contain matching rows and then reads only those granules. The remaining granules are skipped without touching the underlying data files.

The project table uses ORDER BY (region, order_date, user_id). When a query filters on the first two columns—for example, region = 'Asia' AND order_date BETWEEN '2023-06-01' AND '2023-06-30'—the predicate aligns with the key prefix, so the engine narrows the read to a small set of granules. By contrast, a filter on city = 'Tokyo' does not involve the ORDER BY key, so the sparse index cannot effectively prune the scan and every granule must be checked.

Feature 16 runs both queries back-to-back on shard 1. The EXPLAIN PLAN output shows that the primary-key-aligned query reads 5 out of 163 granules (3.1%), while the city query reads all 163 (100%). This difference shows how strongly sparse-index pruning can reduce I/O on a single-month partition.

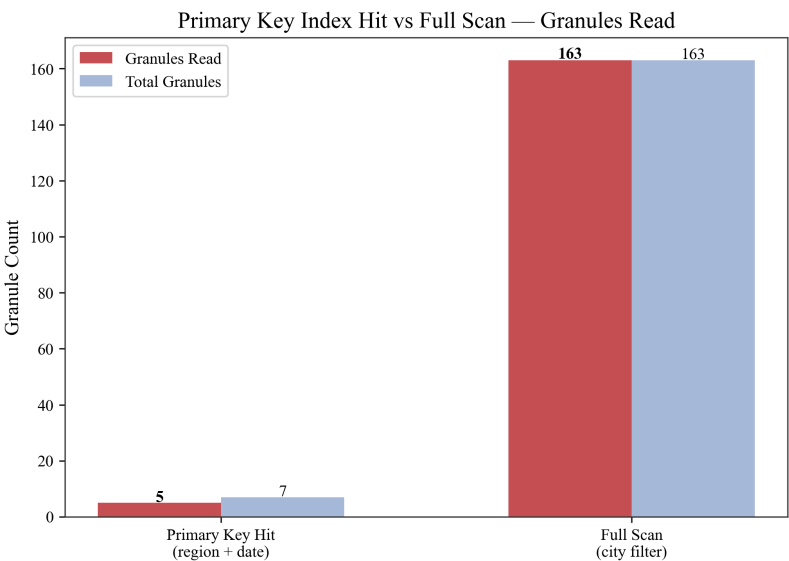


Figure 3. Granules read with primary-key filter vs. full scan

Two skip indexes add a second pruning layer on top of the sparse index. A bloom filter index on `user_id` with granularity 4 allows the engine to test, for each group of four granules, whether a target user ID could be present without decompressing the actual column data. If the bloom filter returns a negative result, that group can be skipped. A minmax index on price with granularity 4 records the smallest and largest value within each four-granule group. As a result, a range filter such as `price >= 400 AND price <= 500` can drop any group whose maximum value is below 400 or whose minimum value is above 500.

3.3 Query Execution — Vectorized Pipeline

ClickHouse does not iterate row by row. Queries run through a pipeline of operators that pass column chunks (vectors) to each other. `EXPLAIN PIPELINE` output for Feature 1 — regional sales aggregation — shows the chain: `MergeTreeSelect` feeds eight parallel `AggregatingTransform` operators, those merge into `GroupingAggregatedTransform`, then through `MergingSortedTransform`, and out to `ExpressionTransform` for final projection. No full-row materialization happens anywhere in that chain.

Feature 6 shows a different execution-level trade-off. `uniqHLL12(user_id)` counts distinct users with a HyperLogLog sketch — a fixed-size probabilistic structure that takes a few kilobytes of memory regardless of how many distinct values pass through. On the project dataset, HyperLogLog returned 101,317 while the exact function `uniqExact` returned 99,989. That's a 1.33% error. The exact version allocates a hash set that grows with the number of distinct user IDs; HyperLogLog does not.

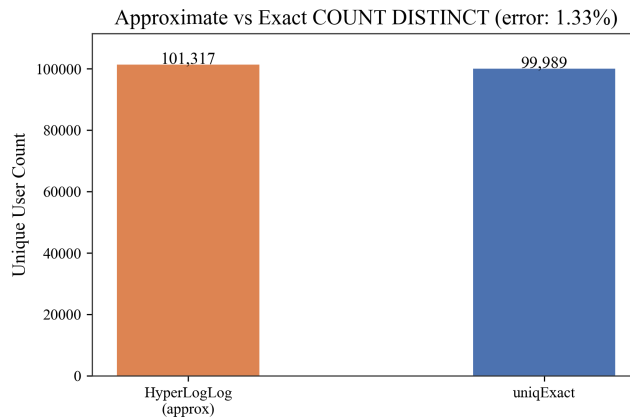


Figure 4. HyperLogLog approximate count vs. exact count of distinct users

3.4 Distribution — Sharding and the Distributed Table Engine

The Docker Compose file starts two ClickHouse containers (shard 1 on port 8123, shard 2 on port 8124) and one ZooKeeper container for coordination. Each shard runs a ReplicatedMergeTree local table called `orders_local`. A Distributed table called `orders` sits on top and handles routing: inserts get hashed by `xxHash64(user_id)` to pick a shard, and reads fan out to both shards at the same time.

After loading one million rows, shard 1 holds 501,515 (50.2%) and shard 2 holds 498,485 (49.8%). This is close to an even split, indicating that the hash function distributes user IDs without noticeable skew.

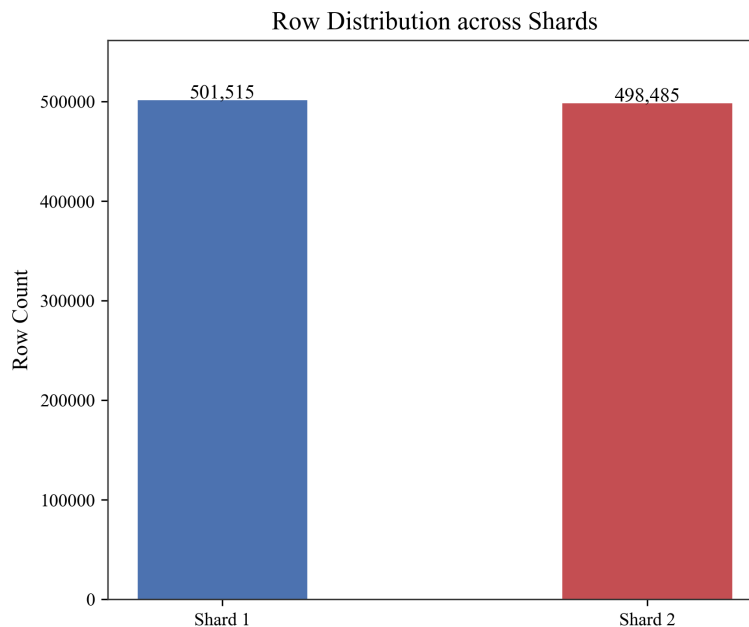


Figure 5. Row count per shard after loading one million rows

For a GROUP BY query (Feature 9), the two shards each run the aggregation locally and send back grouped rows rather than raw data to the coordinator. The coordinator merges those partials into the final answer. EXPLAIN PIPELINE output shows a ReadFromRemote node for shard 2's results feeding into GroupingAggregatedTransform next to the local shard 1 results. That two-phase pattern keeps network traffic low: instead of shipping millions of raw rows, each shard ships a handful of aggregated rows.

4. Application Design

The application (`app.py`) is a Python CLI connecting through the `clickhouse-connect` library. A numbered menu lists sixteen features. The user selects a feature, enters optional parameters such as a date range, user ID, or price bounds, and then sees the query result together with wall-clock execution time. For features intended to highlight internal behavior—such as index pruning, partition skipping, and pipeline shape—EXPLAIN output is displayed above the query result so that both are visible at once.

Schema creation is handled by `schema.py`, which sends DDL to shard 1 with `ON CLUSTER` so that both shards receive the table definition. Data generation is handled by `generate_data.py`, which inserts one million rows in batches of 50,000 through the Distributed table. Five shell commands listed in the README bring up the cluster, create the tables, load the data, and start the application.

Feature 1 aggregates sales by region. Because the data was generated with uniform random sampling, all six regions fall within a narrow range of about \$229–231 million.

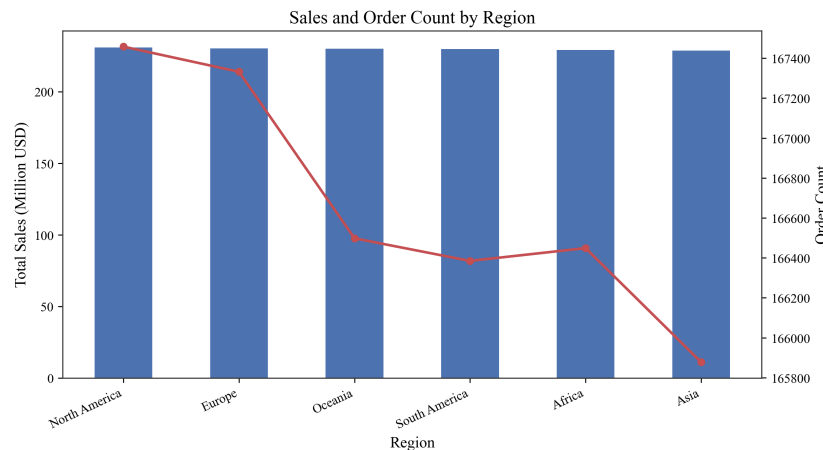


Figure 6. Total sales and order count by region

Feature 4 aggregates monthly revenue. February values dip slightly because those months contain fewer calendar days, while the remaining months stay near \$57 million each.

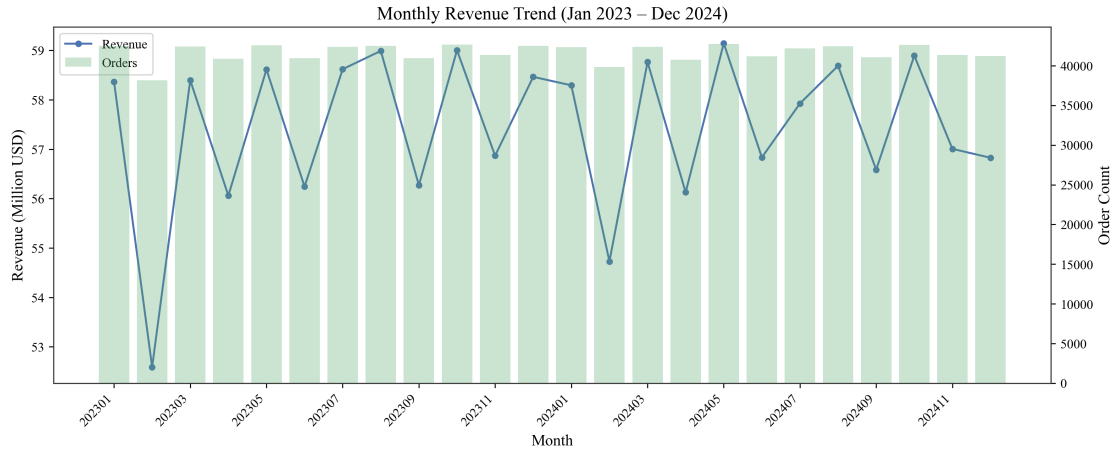


Figure 7. Monthly revenue trend from January 2023 to December 2024

Feature 5 breaks sales down by product category. Eight categories, each capturing roughly one-eighth of total orders.

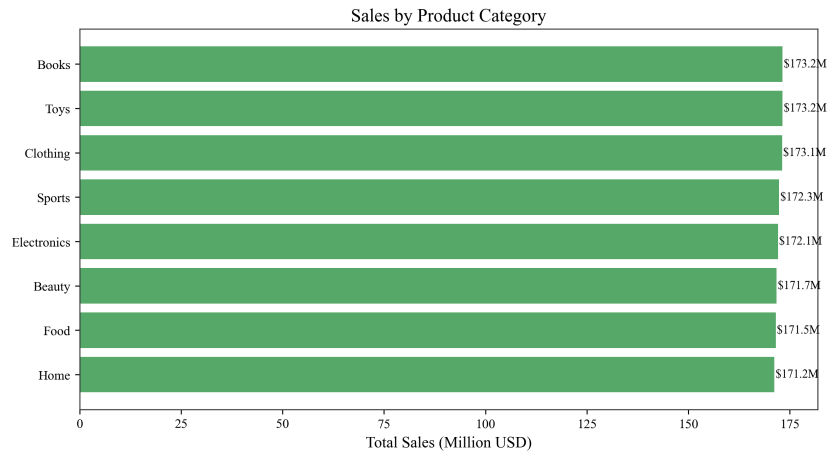


Figure 8 Total sales by product category

5. Mapping Internals to Application Behavior

The table below lists all sixteen features together with what happens inside ClickHouse when the query runs, and why that internal behavior matters.

Table 1. Full mapping between application features and ClickHouse internals

#	Feature	What the App Does	What ClickHouse Does	Why It Matters

1	Regional sales	Groups by region, sums price × quantity	Reads only region, price, quantity column files; eight parallel AggregatingTransform operators process vectors of 8 192 values	Skips eight unused columns; vectorized loops keep CPU caches warm
2	Time-range query	Fetches orders between two dates	Checks partition metadata, opens only the partitions whose month overlaps the range, then binary-searches the sparse index on order_date within surviving partitions	A one-month query drops 23 of 24 partitions before reading any data
3	User lookup	Finds orders for one user_id	Bloom filter skip index tests each four-granule group; groups that cannot contain the target ID are dropped	Avoids decompressing granule groups that don't hold the user
4	Monthly trend	Revenue by toYYYYMM(order_date)	Each partition already maps to one month, so the engine scans partition by partition	Partition-aligned grouping removes the need for a sort step
5	Category stats	Groups by category: sum, avg, count, uniqExact	Reads category, price, quantity, user_id columns; uniqExact builds a hash set per category	Reads four columns out of eleven; hash set gives the true count
6	Approx vs exact distinct	uniqHLL12 side by side with uniqExact	HyperLogLog sketch takes a few KB; hash	1.33% error in exchange for fixed memory — worth

			set grows with distinct count	it when distinct counts get large
7	Top spenders	Ranks users by total spend, top 20	Aggregates grouped by user_id, sorts, returns top 20	Only three columns read; sort runs in memory on a small result set
8	Price range	Filters orders in a price band	MinMax skip index drops four-granule groups whose range doesn't overlap the filter	Avoids reading groups where no matching price can exist
9	Cross-shard agg	Groups by region + category across both shards	Each shard pre-aggregates locally, sends grouped rows; coordinator merges via MergingAggregated	Ships aggregates, not raw rows, across the network
10	Batch insert	Inserts 1 000 new rows	Sorts the batch by ORDER BY, writes one new immutable part; background merge folds it in later	Append-only write path; no in-place updates; merge compacts asynchronously
11	Partition stats	Row count and disk size per partition	Reads system.parts metadata	Shows how PARTITION BY splits data into monthly chunks
12	Column sizes	Compressed/uncompressed bytes per column	Reads system.parts_columns counters	Makes per-column compression ratios visible
13	Cluster info	Lists shards and replicas	Reads system.clusters	Confirms cluster topology: two shards, one replica each

14	Shard distribution	Row count per shard	Connects to each shard, runs count() on the local table	Confirms hash-based routing distributes rows without skew
15	Query stats	Last five SELECT queries with rows read and duration	Reads system.query_log	Shows actual I/O and timing for recent queries
16	Index vs full scan	Two COUNT queries: one on key columns, one on non-key	EXPLAIN shows granule counts; key query reads 5/163, non-key reads 163/163	Puts a number on the cost gap between sparse-index hit and full scan

6. Comparison with MySQL and MongoDB

This section compares ClickHouse with MySQL and MongoDB in terms of storage, indexing, writes, and sharding.

Storage layout. InnoDB packs all columns of a row into a 16 KB page organized by a clustered B+ tree on the primary key. Reading any column means reading the whole page. MongoDB does something similar: each document is a BSON object holding all fields, and a collection scan touches every field in every document. ClickHouse, by contrast, stores columns in separate files, so a query that aggregates only two columns reads only those two columns rather than the full row (Abadi et al., 2008).

Index structure. InnoDB uses a dense B+ tree with one entry per row, and secondary indexes store the primary key as a pointer back to the clustered index. MongoDB similarly uses B-tree indexes with one entry per document. ClickHouse uses a sparse index with one entry per 8,192 rows. This makes the index much smaller, but it is most effective when predicates align with the ORDER BY prefix. For filters on unrelated columns, skip indexes may help, but otherwise the query may require a much broader scan.

How writes work. InnoDB updates pages in a buffer pool, records changes in a write-ahead log, and flushes dirty pages in the background, which makes single-row updates efficient. MongoDB also supports in-place document updates with journaling. ClickHouse,

in contrast, writes immutable parts and merges them later. Updating even a single row requires a mutation that rewrites the affected parts. As a result, ClickHouse performs well for bulk inserts but is less suitable for write-heavy transactional workloads.

Sharding. MySQL does not include built-in sharding and typically relies on external systems such as Vitess or ProxySQL. MongoDB includes built-in sharding with range- or hash-based chunk distribution and a config server. ClickHouse uses the Distributed table engine to fan out queries to shards defined in configuration. Each shard performs local pre-aggregation for GROUP BY queries, and the coordinator merges the partial results. This two-phase design reduces the amount of data transferred across the network.

Table 2 below puts the three systems next to each other.

***Table 2.** ClickHouse vs. MySQL vs. MongoDB*

Aspect	ClickHouse	MySQL (InnoDB)	MongoDB
Storage layout	One file per column per part	All columns in 16 KB row pages	All fields in BSON documents
Primary index	Sparse, 1 entry / 8 192 rows	Dense B+ tree, 1 entry / row	B-tree, 1 entry / document
Compression unit	Per column, per part	Per page (16 KB)	Per block (snappy/zlib)
Write model	Append immutable parts, merge later	In-place page updates + WAL	In-place document updates + journal
Sharding	Built-in Distributed engine	External (Vitess, ProxySQL)	Built-in chunk-based
Sweet spot	Analytical scans, many rows, few columns	Transactional row-level reads and writes	Flexible-schema document access

7. Limitations and Lessons Learned

The synthetic dataset draws every categorical field from a uniform random distribution, so all six regions, all eight categories, and all thirty cities get roughly equal traffic. Real e-commerce data would likely be much more skewed, with a few categories dominating sales and some regions receiving far more orders than others. Such skew would affect several behaviors at once: compression ratios would increase for frequent values, partition sizes would become less even, and hash-based shard routing might concentrate hot users on one shard. Running the same application on a skewed dataset would be a useful direction for future work.

One unexpected problem came from ClickHouse's part format. When parts are small (below a configurable byte and row threshold), the engine stores them in "compact" format, which bundles all columns into a single file. Compact parts do not report per-column sizes in `system.parts_columns` — the counters just read zero. Setting `min_bytes_for_wide_part = 0` forced wide format (one file per column) and gave back the compression numbers. That setting would not be a good default in production, where compact format saves file descriptors, but for a project that needs to expose per-column compression, it's the right trade-off.

The sparse index is strong for range queries that line up with the `ORDER BY` prefix. It offers nothing for filters on columns outside that prefix. Bloom filter and minmax skip indexes help, but they operate on multi-granule groups, not single rows. A workload dominated by point lookups on random columns would be better off on a row-store with dense secondary indexes.

Running only two shards is enough to show that the Distributed engine works and that hash-based routing splits rows evenly. It does not test what happens when shard counts grow, when one shard goes down mid-query, or when memory pressure forces spilling during a large cross-shard aggregation. Expanding to four or eight shards and injecting failures would make the distribution analysis more realistic.

8. Project Link

A **GitHub repository link** containing all project code, documentation, datasets, and related files: <https://github.com/xhuang21/ecommerce-analytics>

References

Schulze, R., Schreiber, T., Yatsishin, I., Dahimene, R., & Milovidov, A. (2024).

ClickHouse—Lightning fast analytics for everyone. *Proceedings of the VLDB Endowment*, 17(12), 3731–3744.

Abadi, D. J., Madden, S. R., & Hachem, N. (2008). Column-stores vs. row-stores: How different are they really? In *Proceedings of the ACM SIGMOD International Conference on Management of Data* (pp. 967–980).